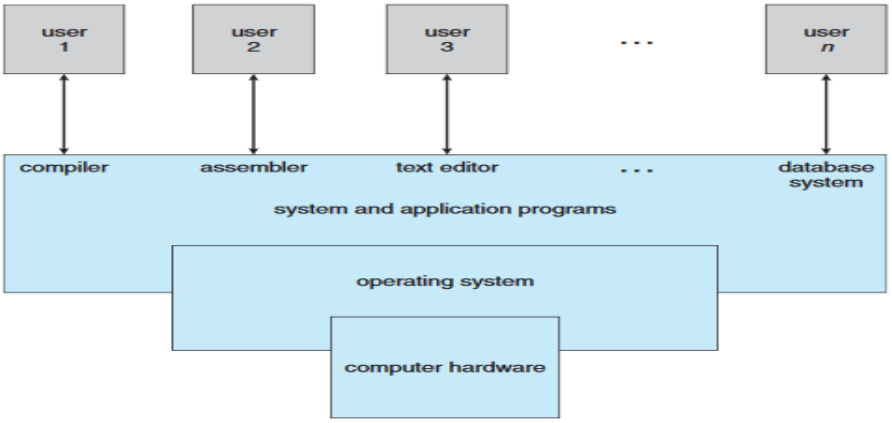
UNIT I
Computer System and Operating System Overview

1.1 Introduction

An operating system is a program that manages a computer's hardware. It also provides a basis for application programs and acts as an intermediary between the computer user and the computer hardware. An amazing aspect of operating systems is how they vary in accomplishing these tasks. Mainframe operating systems are designed primarily to optimize utilization of hardware. Personal computer (PC) operating systems support complex games, business applications, and everything in between. Operating systems for mobile computers provide an environment in which a user can easily interface with the computer to execute programs. Thus, some operating systems are designed to be convenient, others to be efficient, and others to be some combination of the two.



A computer system can be divided roughly into four components: the hardware, the operating system, the application programs, and the users.

- The hardware—the central processing unit(CPU), the memory, and the input/output(I/O)devices— provides the basic computing resources for the system.
- The application programs— such as word processors, spreadsheets, compilers, and Web browsers — define the ways in which these resources are used to solve users' computing problems.
- The operating system controls the hardware and coordinates its use among the various application programs for the various users.

1.2 Operating system Structures

A system as large and complex as a modern operating system must be engineered carefully if it is to function properly and be modified easily. A common approach is to partition the task into small components, or modules, rather than have one monolithic system. Each of these modules should be well defined portion of the system, with carefully defined inputs, outputs and function.

Simple Structure

Many operating systems do not have well-defined structures. Frequently, such systems started as small, simple, and limited systems and then grew beyond their original scope.

MS-DOS is an example of such a system. It was originally designed and implemented by a few people who had no idea that it would become so popular. It was written to provide the most functionality in the least space, so it was not carefully divided into modules.

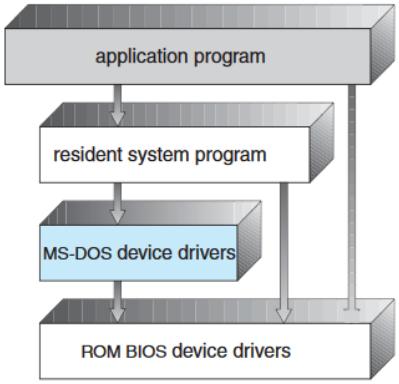


Figure MS-DOS layer structure.

In MS-DOS, the interfaces and levels of functionality are not well separated. For instance, application programs are able to access the basic I/O routines to write directly to the display and disk drives. Such freedom leaves MS-DOS vulnerable to errant (or malicious) programs, causing entire system crashes when user programs fail. Of course, MS-DOS was also limited by the hardware of its era.

Another example of limited structuring is the original UNIX operating system. Like MS-DOS, UNIX initially was limited by hardware functionality.

It consists of two separable parts: the kernel and the system programs. The kernel is further separated into a series of interfaces and device drivers, which have been added and expanded over the years as UNIX has evolved.

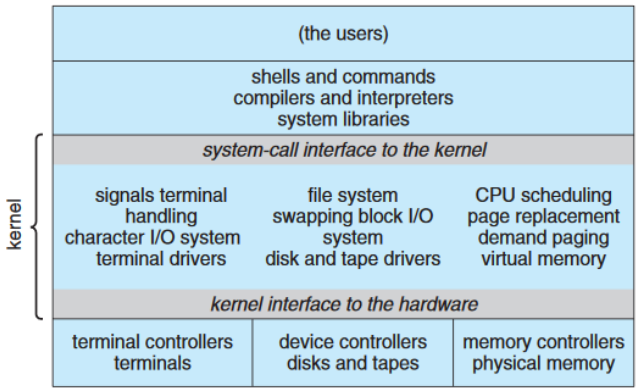
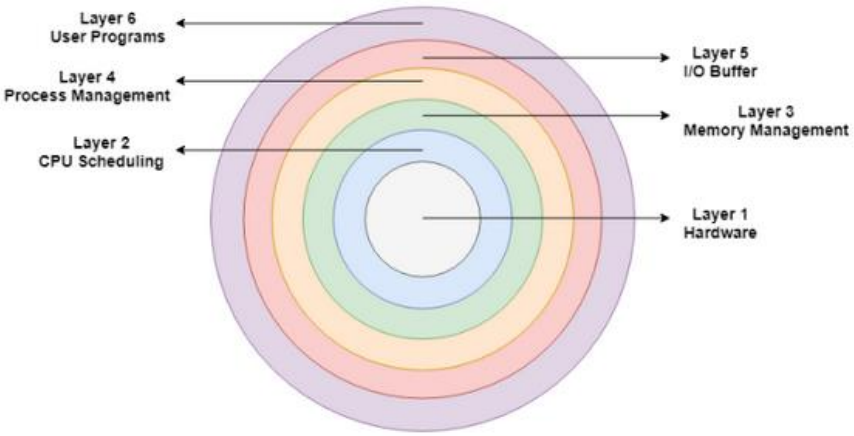


Figure Traditional UNIX system structure.

We can view the traditional UNIX operating system as being layered to some extent. Everything below the system-call interface and above the physical hardware is the kernel.

Layered Approach

A system can be made modular in many ways. One method is the layered approach, in which the operating system is broken into a number of layers (levels). The bottom layer (layer 0) is the hardware, the highest (layer N) is the user interface.



LAYERED OPERATING SYSTEM

An operating-system layer is an implementation of an abstract object made up of data and the operations that can manipulate those data. A typical operating system layer — say, layer M— consists of data structures and a set of routines that can be invoked by higher-level layers. Layer M, in turn, can invoke operations on lower-level layers.

Layering provides a distinct advantage in an operating system. All the layers can be defined separately and interact with each other as required. Also, it is easier to create, maintain and update the system if it is done in the form of layers. Change in one layer specification does not affect the rest of the layers.

Once the first layer is debugged, its correct functioning can be assumed while the second layer is debugged, and so on. If an error is found during the debugging of a particular layer, the error must be on that layer, because the layers below it are already debugged.

Each layer is implemented only with operations provided by lower-level layers. A layer does not need to know how these operations are implemented. It needs to know only what these operations do. Hence, each layer hides the existence of certain data structures, operations, and hardware from higher-level layers.

One problem with the layered structure is that each layer needs to be carefully defined. This is necessary because the upper layers can only use the functionalities of the layers below them.

A final problem with layered implementations is that they tend to be less efficient than other types. For instance, when a user program executes an I/O operation, it executes a system call that is trapped to the I/O layer, which calls the memory-management layer, which in turn calls the CPU-scheduling layer, which is then passed to the hardware. At each layer, the parameters may be modified; data may need to be passed, and so on. Each layer adds overhead to the system call. The net result is a system call that takes longer than does one on a non layered system.

Micro kernels

This method structures the operating system by removing all nonessential components from the kernel and implementing them as system and user-level programs. The result is a smaller kernel. There is little consensus regarding which services should remain in the kernel and which should be implemented in user space. Typically, however, micro-kernels provide minimal process and memory management, in addition to a communication facility. The main function of the microkernel is to provide communication between the client program and the various services that are also running in user space. Communication is provided through message passing.

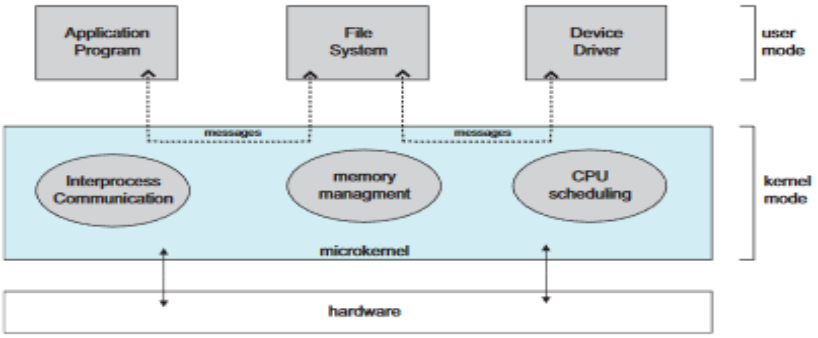


Figure Architecture of a typical microkernel.

For example, if the client program wishes to access a file, it must interact with the file server. The client program and service never interact directly. Rather, they communicate indirectly by exchanging messages with the microkernel.

One benefit of the microkernel approach is that it makes extending the operating system easier. All new services are added to user space and consequently do not require modification of the kernel.

1.3 Batch Operating System

The users of a batch operating system do not interact with the computer directly. Each user prepares his job on an off-line device like punch cards and submits it to the computer operator. The programmers leave their programs with the operator and the operator then sorts the programs with similar requirements into batches. To speed up processing, jobs with similar needs are batched together and run as a group. It is also referred as single user multi-task operating system.

The problems with Batch Systems are as follows −

- Lack of interaction between the user and the job.
- CPU is often idle, because the speed of the mechanical I/O devices is slower than the CPU.
- Difficult to provide the desired priority.

1.4 Multi-programmed

Multiprogramming is an extension to the batch processing where the CPU is kept always busy. Each process needs two types of system time: CPU time and IO time.

In multi-programming, a CPU handles different programs in its main memory at a time. The processor switches from a job to job as needed and keeps the jobs advancing. In other words the multi-programming operating system can handle two or more jobs at a time in execution or states of execution. Its advantage is that the CPU time is not wasted during the following occasions:

- Job is completed
- Error is detected
- I/O is required
- Job current time slice run out.

Maximum utilization of the processor, memory and other I/O devices is possible using multiprogramming.

1.5 Time-shared

Time-sharing or multitasking is a method of allocating processor time. Using timesharing the processor time is divided into time slices. Each job is assigned a particular time slice in which processors attends to that job and executes it. After the completion of that time slice the

processor keeps the status of executes job and takes the execution of next job even the first job is not completed.

The main difference between Multi-programmed Batch Systems and Time-Sharing Systems is that in case of Multi-programmed batch systems, the objective is to maximize processor use, whereas in Time-Sharing Systems, the objective is to minimize response time.

Multiple jobs are executed by the CPU by switching between them, but the switches occur so frequently. Thus, the user can receive an immediate response. For example, in a transaction processing, the processor executes each user program in a short burst or quantum of computation. That is, if **n** users are present, then each user can get a time quantum. When the user submits the command, the response time is in few seconds at most.

The operating system uses CPU scheduling and multiprogramming to provide each user with a small portion of a time. Computer systems that were designed primarily as batch systems have been modified to time-sharing systems.

Advantages of Timesharing operating systems are as follows −

- Provides the advantage of quick response.
- Avoids duplication of software.
- Reduces CPU idle time.

Disadvantages of Time-sharing operating systems are as follows −

- Problem of reliability.
- Question of security and integrity of user programs and data.
- Problem of data communication.

1.6 Personal Computer

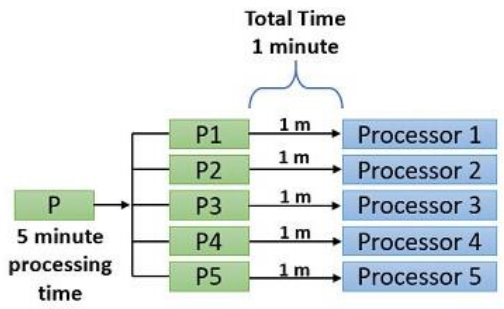
Personal computer operating system provides a good interface to a single user. Personal computer operating systems are widely used for word processing, spreadsheets and Internet access. Personal computer operating system is made only for personal.

You can say that your laptops, computer systems, tablets etc. are your personal computers and the operating system such as windows 7, windows 10, android, etc. are your personal computer operating system.

And you can use your personal computer operating system for your personal purposes, for example, to chatting with your friends using some social media sites, reading some articles from internet, making some projects through Microsoft PowerPoint or any other, designing your website, programming something, watching some videos and movies, listening to some songs and many more.

1.7 Parallel Processing

Parallel processing requires multiple processors and all the processor works simultaneously in the system. Here, the task is divided into subparts and these subparts are then distributed among the available processors in the system. Parallel processing completes the job on the shortest possible time.



All the processors in the parallel processing environment should run on the same operating system. All processors here are tightly coupled and are packed in one casing. All the processors in the system share the common secondary storage like the hard disk. As this is the first place where the programs are to be placed.

1.8 Distributed operating System

Distributes systems are multiprocessor computers in which the computations will be paralleled into sub computations that they can be distributes among the processors. These processors do not share the memory or clock. Each processor has its own local memory. The processors communicate with one another through various communication lines. These systems are usually referred to as “loosely coupled” systems. These processors in a distributed system may vary in size and function. They are referred to as sites, notes, computers etc.

In the networks these distributes computing make use of some special purpose computers also and achieves high speed computation. These distributed systems are inexpensive.

The advantages of distributed systems are as follows −

- With resource sharing facility, a user at one site may be able to use the resources available at another.
- Speedup the exchange of data with one another via electronic mail.
- If one site fails in a distributed system, the remaining sites can potentially continue operating.
- Better service to the customers.
- Reduction of the load on the host computer.
- Reduction of delays in data processing

1.9 Real Time operating System

A real-time system is defined as a data processing system in which the time interval required to process and respond to inputs is so small that it controls the environment. The time taken by the system to respond to an input and display of required updated information is termed as the response time. So in this method, the response time is very less as compared to online processing.

Real time systems are used where there is a need for continuously monitoring of the situation and to supply immediate response. These systems are used dedicated systems and they extend services when needed to respond quickly. They will be busy for a large time in monitoring the status. System that control the scientific experiments, medical imaging systems, industrial control systems and some display systems are real time systems. A real time system is considered to function correctly only if it returns the correct result within any time constraints. These systems have relatively high costs when compared to distributed systems.

There are two types of real-time operating systems.

Hard real-time systems

Hard real-time systems guarantee that critical tasks complete on time. In hard real-time systems, secondary storage is limited or missing and the data is stored in ROM. In these systems, virtual memory is almost never found.

Soft real-time systems

Soft real-time systems are less restrictive. A critical real-time task gets priority over other tasks and retains the priority until it completes. Soft real-time systems have limited utility than hard real-time systems. For example, multimedia, virtual reality, Advanced Scientific Projects likes undersea exploration and planetary rovers, etc.

1.10 System components

Some common components of operating system include:

1. Process Management
2. Main Memory Management
3. File Management
4. I/O System Management
5. Secondary storage Management
6. Protection System

1. Process Management

A process is a unit of work in a system/program in execution. A system consists of a collection of processes. All these processes can potentially execute concurrently, by multiplexing the CPU among them. Process management involves the following activities:

- ✓ Creation and deletion of user and system processes.
- ✓ Suspension and resumption of processes.
- ✓ Provision of mechanisms for process synchronization
- ✓ Provision of mechanisms for process communication
- ✓ Provision of mechanisms for deadlock handling

2. Main memory management

Memory is a large array of words or bytes, each with its own address. It is a repository of quickly accessible data shared by the CPU and I/O devices. Main memory is a volatile storage device. It loses its contents in the case of system failure. For a program to be executed, it should be first loaded in the main memory.

The operating system is responsible for the following activities in connections with memory management:

- ✓ Keep track of which parts of memory are currently being used and by whom.
- ✓ Decide which processes to load when memory space becomes available.
- ✓ Allocate and de-allocate memory space as needed

3. File Management

A file system is organized into directories for efficient or easy navigation and usage. These directories may contain other directories and other files. An Operating System carries out the following file management activities. It keeps track of where information is stored, user access settings, the status of every file, and more. An OS keeps track of information regarding the creation, deletion, transfer, copy, and storage of files in an organized way.

A file is a collection of related information defined by its creator. Commonly, files represent programs (both source and object forms) and data.

The operating system is responsible for the following activities in connections with file management:

- ✓ Creation and deletion of files.
- ✓ Creation and deletion of directories.
- ✓ Support of primitives for manipulating files and directories.
- ✓ Mapping files onto secondary storage.
- ✓ File backup on stable (nonvolatile) storage media

4. I/O System Management

The I/O system consists of:

- ✓ A buffer-caching system
- ✓ A general device-driver interface
- ✓ Drivers for specific hardware devices

I/O system management allots the device to processes according to their requirement in a scheduled manner.

5. Secondary-Storage Management

Since main memory (primary storage) is volatile and too small to accommodate all data and programs permanently, the computer system must provide secondary storage to back up main memory.

Most modern computer systems use disks as the principle on-line storage medium, for both programs and data.

The operating system is responsible for the following activities in connection with disk management:

- ✓ Free space management
- ✓ Storage allocation
- ✓ Disk scheduling

6. Protection System

Protection refers to a mechanism for controlling access by programs, processes, or users to both system and user resources.

The protection mechanism must

- ✓ Distinguish between authorized and unauthorized usage.
- ✓ specify the controls to be imposed.
- ✓ Provide a means of enforcement.

1.11 Operating System services

An operating system provides an environment for the execution of programs. It provides certain services to programs and to the users of those programs. Services provided are differ from one operating system to another, but here we identify common classes for convenience and make easier to programmer.

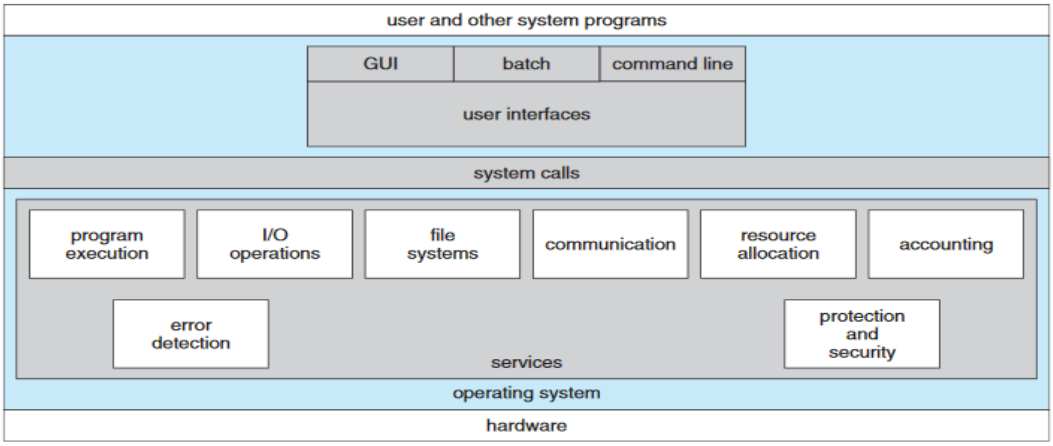


Figure A view of operating system services.

- One set of operating system services provides functions that are helpful to the user.

User interface

Almost all operating systems have a user interface (UI). This interface can take several forms.

- One is a command-line interface (CLI), which uses text commands and a method for entering them.
- Another is a batch interface, in which commands and directives to control those commands are entered into files, and those files are executed.
- Most commonly, a graphical user interface (GUI) is used.

Program execution

The system must be able to load a program into memory, run a program. The program must be able to end its execution, either normally or abnormally (indicating error).

I/O operations

A running program may require I/O, which may involve a file or an I/O device. For specific devices, special functions may be desired. For efficiency and protection, users usually cannot control I/O devices directly. Therefore, the operating system must provide a means to do I/O.

File-system manipulation

Programs need to read and write files and directories. They also need to create and delete them by name, search for a given file, and list file information. Finally, some operating systems include permissions management to allow or deny access to files or directories based on file ownership.

Many operating systems provide a variety of file systems, sometimes to allow personal choice and sometimes to provide specific features or performance characteristics.

Communication

One process needs to exchange information with another process that are executing on the same computer on different computer systems tied together by a computer network. Communications may be implemented via shared memory, in which two or more processes read and write to a shared section of memory, or message passing, in which packets of information in predefined formats are moved between processes by the operating system.

Error detection

The operating system needs to be detecting and correcting errors constantly that may occur in the CPU and memory hardware, in I/O devices and in the user program. For each type of error, the operating system should take the appropriate action to ensure correct and consistent computing. Sometimes, it has no choice but to halt the system. At other times, it might terminate an error-causing process or return an error code to a process for the process to detect and possibly correct.

- Another set of operating system functions exists not for helping the user but rather for ensuring the efficient operation of the system itself.

Resource allocation

When there are multiple users or multiple jobs running at the same time, resources must be allocated to each of them. The operating system manages many different types of resources. Operating systems have CPU-scheduling routines that take into account the speed of the CPU, the jobs that must be executed, the number of registers available, and other factors.

Accounting

We want to keep track of which users use how much and what kinds of computer resources.

Protection and security

The information stored in a multiuser or networked computer system may want to control use of that information. When several separate processes execute concurrently, it should not be possible for one process to interfere with the others or with the operating system itself.

Protection involves ensuring that all access to system resources is controlled. Security starts with requiring each user to authenticate him/her to the system, usually by means of a password, to gain access to system resources.

1.12 System Calls

The interface between a process and an operating system is provided by system calls. In general, system calls are available as assembly language instructions. They are also included in the manuals used by the assembly level programmers. System calls are usually made when a process in user mode requires access to a resource. Then it requests the kernel to provide the resource via a system call.

Why do you need System Calls in OS

- Reading and writing from files demand system calls.
- If a file system wants to create or delete files, system calls are required.
- System calls are used for the creation and management of new processes.
- Network connections need system calls for sending and receiving packets.
- Access to hardware devices like scanner, printer, need a system call.

System calls can be grouped roughly into five major categories:

- Process control
- file management
- device management
- information maintenance
- Communication and protection

Process control

- ✓ These system calls deal with processes such as process creation, process termination etc

    - end, abort
    - load, execute
    - create process, terminate process
    - get process attributes, set process attributes
    - wait for time
    - wait event, signal event
    - allocate and free memory

A running program needs to be able to halt its execution either normally (end ()) or abnormally (abort ()).

A process or job executing one program may want to load() and execute()another program. This feature allows the command interpreter to execute a program as directed by.

Here is a system call specifically for this purpose (create process() or submit job()).

If we create a new job or process, or perhaps even a set of jobs or processes, we should be able to control its execution. This control requires the ability to determine and reset the attributes of a job or process, including the job's priority, its maximum allowable execution time, and so on (getprocessattributes() and setprocessattributes()).

We may also want to terminate a job or process that we created (terminate process()) if we find that it is incorrect or is no longer needed.

Having created new jobs or processes, we may need to wait for them to finish their execution. We may want to wait for a certain amount of time to pass (wait time ()).

More probably, we will want to wait for a specific event to occur (wait event ()). The jobs or processes should then signal when that event has occurred (signal event ()).

File management

- ✓ These system calls are responsible for file manipulation such as creating a file, reading a file, writing into a file etc.

    - create file, delete file
    - open, close
    - read, write, reposition
    - get file attributes, set file attributes

We first need to be able to create ()and delete() files. Either system call requires the name of the file and perhaps some of the file's attributes. Once the file is created, we need to open () it and to use it. We may also read (),write(), or reposition() (rewind or skip to the end of the file, for example).Finally, we need to close()the file, indicating that we are no longer using it.

Device management

- ✓ These system calls are responsible for device manipulation such as reading from device buffers, writing into device buffers etc.

    - request device, release device
    - read, write, reposition
    - get device attributes, set device attributes
    - logically attach or detach devices

A system with multiple users may require us to first request () a device, to ensure exclusive use of it. After we are finished with the device, we release () it. Once the device has been requested (and allocated to us), we can read (),write(), and (possibly) reposition()the device, just as we can with files.

Information maintenance

- ✓ These system calls handle information and its transfer between the operating system and the user program.

    - get time or date, set time or date
    - get system data, set system data
    - get process, file, or device attributes
    - set process, file, or device attributes

Many system calls exist simply for the purpose of transferring information between the user program and the operating system. For example, most systems have a system call to return the current time () and date (). Other system calls may return information about the system, such as the number of current users, the version number of the operating system, the amount of free memory or disk space, and so on. Generally, calls are also used to reset the process information (getprocessattributes () and setprocessattributes ())

Communications

- ✓ These system calls are useful for inter-process communication. They also deal with creating and deleting a communication connection.

    - create, delete communication connection
    - send, receive messages
    - transfer status information
    - attach or detach remote devices

general purpose open() and close()calls provided by the file system or to specific open connection() and close connection()system calls, depending on the system's model of communication. The recipient process usually must give its permission for communication with an accept connection ()call. They execute await for connection ()call and are awakened when a connection is made. The source of the communication, known as the client, and the receiving daemon, known as a server ,then exchange messages by using read message() and write message() system calls. The close connection ( ) call terminates the communication.